



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Технологии и инструментарий анализа больших данных»

Практическая работа №4

**Выполнение задач обработки данных
в распределённых вычислениях**

Студент группы ИНБО-12-23, Албахтин И.В.

(подпись)

Преподаватель Воронцов Ю.А.

(подпись)

Москва 2026 г.

Содержание

1 Цель и условия эксперимента	3
2 Стенд и источники	3
3 Ленивые вычисления	4
4 Repartition, coalesce и groupBy	4
5 Стратегии JOIN.....	6
6 Persist и полная стоимость повторного использования.....	7
7 Benchmark, shuffle partitions и файлы.....	8
8 Самостоятельный вариант В.....	10
9 Выводы и ограничения	12
Ответы на контрольные вопросы	13
Источники и приложения	18

1 Цель и условия эксперимента

Исследованы ленивое вычисление, разделы данных, shuffle, стратегии JOIN, persist, повторные измерения и число выходных файлов. Основные опыты используют Olist; самостоятельный вариант В соединяет order_items и sellers и агрегирует по seller_state. Все времена и счётчики получены заново в личной VM.

2 Стенд и источники

Результаты приведены в таблице 1.

Таблица 1 – Паспорт личной виртуальной машины

Параметр	Фактическое значение
VM	TIABD-Albakhtin-IV
OC	Ubuntu Server 22.04.5 LTS
Ресурсы	2 vCPU; 4 ГБ RAM; независимый VDI 40 ГБ
Python	3.10.12
Java	openjdk version "17.0.20.1" 2026-08-18
Spark	3.5.7
Master / parallelism	local[*] / 2
Shuffle partitions / AQE	8 / false

Работа выполнена в новой VM с отдельной установкой Ubuntu и пользователем albakhtin. Каждый ноутбук запущен полностью в новом ядре. Local[*] использует два виртуальных ядра одного узла; физическая сеть кластера в таком режиме не проверяется. Методичка рекомендует Python 3.11; системный Python 3.10 совместим с выбранными Spark 3.5.7 и JDK 17.

В основной серии AQE=false, SQL shuffle partitions=8, autoBroadcastJoinThreshold=-1 и driver memory=1536 МБ. Для каждого повторяемого опыта выполнен прогрев и три измерения; подготовка SQLite, CSV и Parquet исключена из измеряемых действий. Фоновые контейнеры и нагрузка хоста остаются ограничением.

Результаты приведены в таблице 2.

Таблица 2 – Входные Parquet DataFrame

Таблица	Строки	Партиции
geolocation	1 000 163	2
order_items	112 650	2
products	32 951	1
sellers	3 095	1
product_category_name_translation	71	1

3 Ленивые вычисления

Списки Jobs до и после select/filter/withColumn пусты. После count появились Jobs [26]; получено 1 000 163 строк за 0,142 с. Count может исключать вычисление неиспользуемого нового столбца, поэтому замер не описывает стоимость всех выражений цепочки.

Результаты приведены в таблице 3.

Таблица 3 – Наиболее частые штаты geolocation

Штат	Строки	Доля %
SP	404 268	40,42
MG	126 336	12,63
RJ	121 169	12,11
RS	61 851	6,18
PR	57 859	5,78

4 Repartition, coalesce и groupBy

Данные сначала разбиты на 16 разделов по штату, затем сведены к четырём. Repartition выполняет новое перераспределение, coalesce объединяет существующие разделы. Наличие Coalesce не отменяет исходный Exchange; структуру следует оценивать по всему физическому плану.

Результаты приведены в таблице 4.

Таблица 4 – Фактическая нагрузка четырёх разделов

Способ	Минимум	Максимум	Среднее
repartition4	250 040	250 041	250 040,75
coalesce4	88 462	626 485	250 040,75

На рисунке 1 представлен результат анализа.

```
TIABD-Albakhtin-IV | student: Albakhtin I.V. | fresh run

TIABD PRACTICE 4 | PARTITIONS
variant      min      max      mean
skewed16      1301    466119   83346.92
repartition4   250040  250041   250040.75
coalesce4      88462   626485   250040.75
Lazy: {'before': [], 'after_transform': [], 'after_action': [26], 'rows': 1000163, 'seconds': 0.14226042600000071}
```

Рисунок 1 – Нагрузки разделов и журнал ленивого действия в новой VM

Группировка по seller_id имеет частичную и финальную HashAggregate с Exchange между ними. Filter и Project являются локальными преобразованиями; Exchange меняет распределение. Sort добавляет сортировку, а запись и count инициируют выполнение.

Результаты приведены в таблице 5.

Таблица 5 – Счётчики измеренных действий Spark UI

Опыт	Jobs	Stages	Tasks	Read КиБ	Write КиБ
GroupBy	1	2	10	262,96	262,96
Repartition 4	1	4	23	172,58	172,58
Coalesce 4	1	3	7	151,84	151,84
SMJ	1	4	12	3 825,50	3 825,50
ВНЖ	2	3	4	0,12	0,12
Материализация	3	4	5	0,12	0,12
Кэш действие	1	3	11	6,29	6,29

5 Стратегии JOIN

Order_items и products соединены по product_id. Обе стратегии возвращают 112 650 строк. SMJ имеет распределение и сортировку по ключу; ВНЖ передаёт малую сторону через BroadcastExchange. Небольшой обмен финального count может сохраняться и при ВНЖ.

Результаты приведены в таблице 6.

Таблица 6 – Первый count и медианы трёх повторений JOIN

Стратегия	Первый count с	Медиана с
SMJ	0,338	0,218
ВНЖ	0,284	0,135

На рисунке 2 представлен результат анализа.

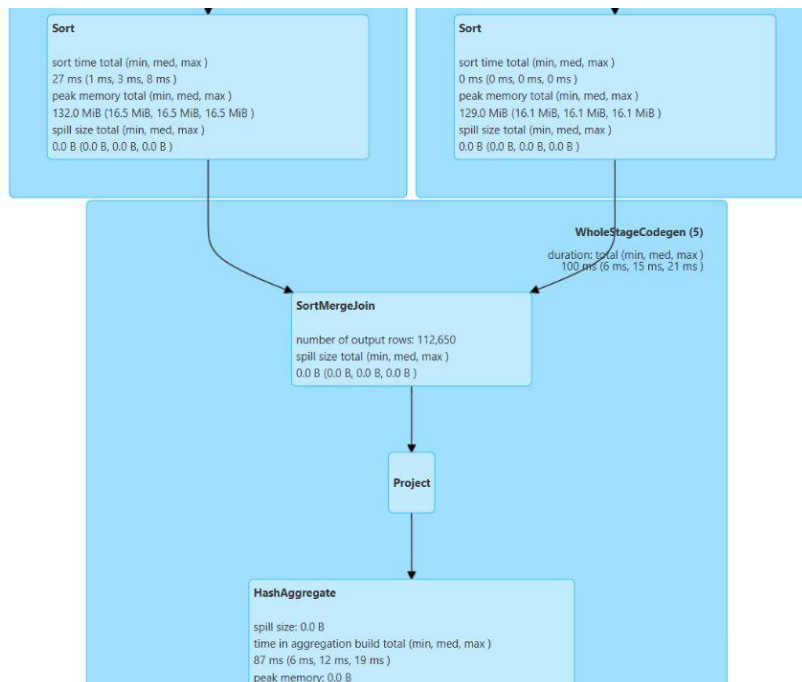


Рисунок 2 – Физический план Sort Merge Join живого приложения

На рисунке 3 представлен результат анализа.

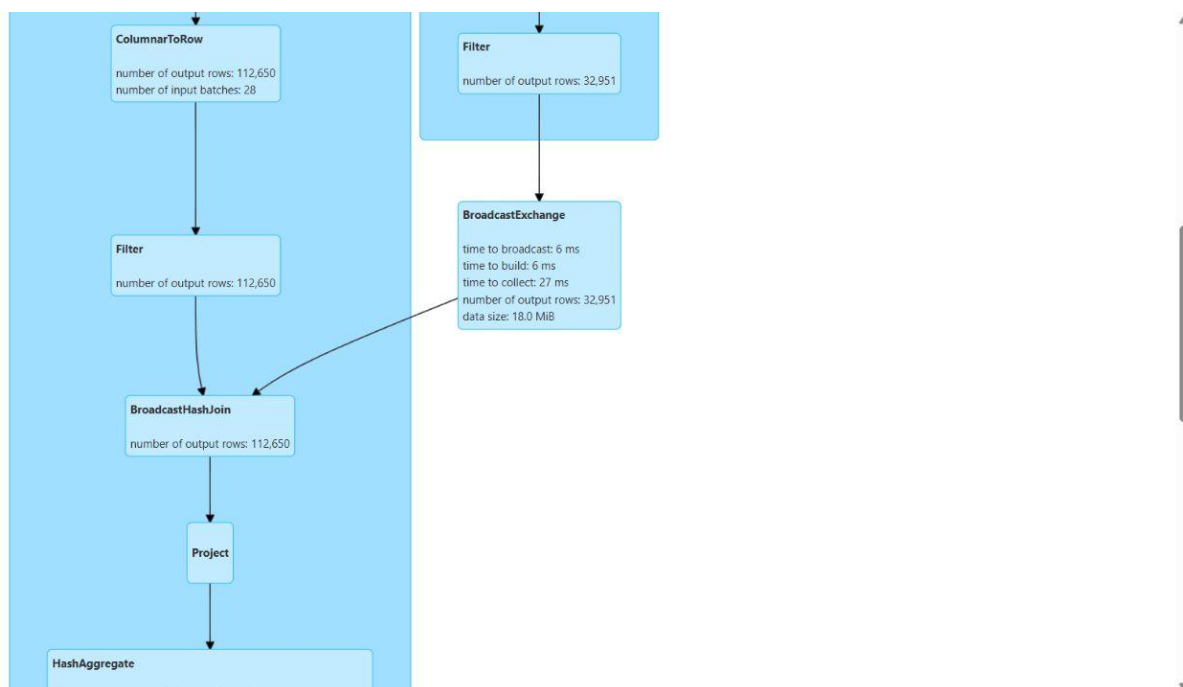


Рисунок 3 – Физический план Broadcast Hash Join живого приложения

6 Persist и полная стоимость повторного использования

Результаты приведены в таблице 7.

Таблица 7 – Стоимость двух действий

Вариант	Материализация с	Повторные действия с	Итого с
Без кэша	—	0,665	0,665
С кэшем	0,663	0,331	0,994

Использован MEMORY_AND_DISK. Контрольные суммы до и после persist совпадают с допуском 0,01. Повторные действия заняли 0,331 с; полная стоимость с материализацией — 0,994 с. Кэш не окупился за два действия в этой серии. После опыта выполнен unpersist(blocking=True).

На рисунке 4 представлен результат анализа.

ID	RDD Name	Storage Level	Cached Partitions	Fraction Cached	Size in Memory	Size on Disk
476	*(3) Project [seller_id#206, product_id#205, order_id#203, order_item_id#204, shipping_limit_date#207, price#208, freight_value#209, product_category_name#218, seller_state#238, (price#208 + freight_value#209) AS gross_value#710] +- *(3) BroadcastHashJoin [seller_id#206], [seller_id#235], LeftOuter, BuildRight, false :- *(3) Project [product_id#205, order_id#203, order_item_id#204, seller_id#206, shipping_limit_date#207, price#208, freight_value#209, product_category_name#218] : +- *(3) BroadcastHashJoin [product_id#205], [product_id#217], LeftOuter, BuildRight, false :- *(3) ColumnarToRow :: +- FileScan parquet [order_id#203,order_item_id#204,product_id#205,seller_id#206,shipping_limit_date#207,price#208,freight_value#209] Batched: true, DataFilters: [], Format: Parquet, Location: InMemoryFileIndex(1 paths)[file:/home/albakhtin/tiabd/АлбакhtiНВ/Spark/data/spark_input_parquet/o..., PartitionFilters: [], PushedFilters: [], ReadSchema: struct<order_id:string,order_item_id:int,product...	Disk Memory Serialized 1x Replicated	2	100.00%	8.3 MiB	0.0 B

Рисунок 4 – Фактическая материализация кэша в Spark Storage

7 Benchmark, shuffle partitions и файлы

Синтетический набор содержит 2 000 000 строк; $key=id\%1000$, $value=\sqrt{id+1}$. Для $p=1,2,4,8,16,32$ выполнены по три замера после прогрева. Измеряется action с агрегацией, а не время построения DataFrame. Допуск контрольных сумм учитывает порядок сложения double.

Результаты приведены в таблице 8.

Таблица 8 – Повторения синтетического benchmark

p	t1 с	t2 с	t3 с	Медиана с	T1/Tr
1	0,133	0,055	0,059	0,059	1,000
2	0,195	0,090	0,104	0,104	0,570
4	0,132	0,090	0,102	0,102	0,581
8	0,212	0,112	0,109	0,112	0,528
16	0,226	0,139	0,178	0,178	0,333
32	0,220	0,161	0,159	0,161	0,366

Минимальная медиана этой серии получена при $p=1$: 0,059 с. На двух vCPU рост числа разделов не гарантирует ускорения; результат нельзя переносить на большой физический кластер.

Результаты приведены в таблице 9.

Таблица 9 – Отдельные одиночные опыты *shuffle partitions*

q	Группы	Время с
2	27	0,341
4	27	0,213
8	27	0,282
16	27	0,278
32	27	0,281

Результаты приведены в таблице 10.

Таблица 10 – Фактические выходные *Parquet* файлы

p	Файлы	Объём МиБ	Средний КиБ
1	1	6,456	6 611,425
4	4	7,464	1 910,897
16	16	8,756	560,407
64	64	10,301	164,824

Запись действительно создаёт part-файлы; их число и размеры измерены после завершения write. При AQE=true исходные 32 *shuffle partitions* дали 27 групп и 1 выходную партицию; итоговый план содержит адаптивное объединение разделов. После опыта базовая конфигурация восстановлена.

На рисунке 5 представлен результат анализа.

```

TIABD-Albakhtin-IV | student: Albakhtin I.V. | fresh run

TIABD PRACTICE 4 | BENCHMARK
p      t1      t2      t3      median  speedup
1      0.133   0.055   0.059   0.059    1.000
2      0.195   0.090   0.104   0.104    0.570
4      0.132   0.090   0.102   0.102    0.581
8      0.212   0.112   0.109   0.112    0.528
16     0.226   0.139   0.178   0.178    0.333
32     0.220   0.161   0.159   0.161    0.366
files  MiB      mean KiB
1      6.456    6611.425
4      7.464    1910.897
16     8.756    560.407
64     10.301   164.824

```

Рисунок 5 – Новые замеры и размеры *Parquet* файлов в консоли VM

8 Самостоятельный вариант В

Связка `order_items–sellers` исследуется отдельно от основной связки `items–products`. `Select`, `filter(price>0)` и `withColumn(gross)` описывают план. После `action` выполняется `JOIN` по `seller_id`, затем группировка по `seller_state` с подсчётом позиций и суммой цены с доставкой.

Результаты приведены в таблице 11.

Таблица 11 – Первые штаты продавцов по gross

Штат	Позиции	Цена с доставкой
SP	80 342	10 235 883,88
PR	8 671	1 458 900,73
MG	8 827	1 224 159,80
RJ	4 818	937 814,12
SC	4 075	738 973,13

Результаты приведены в таблице 12.

Таблица 12 – Распределение самостоятельных четырёх партиций

Способ	Минимум	Максимум	Среднее
repartition	28 161	28 165	28 162,50
coalesce	5 348	88 381	28 162,50

Результаты приведены в таблице 13.

Таблица 13 – Стратегии JOIN самостоятельной части

Стратегия	Время с	Stages	Tasks	Shuffle КиБ
SMJ	0,192	4	12	1 337,59
BHJ	0,088	3	4	0,12

Два действия без кэша заняли 0,318 с, материализация — 0,198 с, повторные действия — 0,239 с. Общая стоимость с кэшем 0,437 с; он не окупился за эти два действия.

Результаты приведены в таблице 14.

Таблица 14 – Три повторения для р самостоятельной части

р	t1 с	t2 с	t3 с	Медиана с
1	0,115	0,077	0,079	0,079
2	0,135	0,128	0,129	0,129
4	0,157	0,133	0,125	0,133
8	0,133	0,114	0,129	0,129

Результаты приведены в таблице 15.

Таблица 15 – Запись самостоятельного агрегата

Партиции	Файлы	Объём МиБ
1	1	0,0013
4	4	0,0041

На рисунке 6 представлен результат анализа.

spark 3.5.7 Jobs Stages Storage Environment Executors SQL / DataFrame TIABD_Practice_4_Albakhtin_IV application UI						
Storage						
▼ RDDs						
ID	RDD Name	Storage Level	Cached Partitions	Fraction Cached	Size in Memory	Size on Disk
955	*(2) Project [seller_id#206, price#208, freight_value#209, gross#2312, seller_zip_code_prefix#236, seller_city#237, seller_state#238] +- *(2) BroadcastHashJoin [seller_id#206], [seller_id#235], Inner, BuildRight, false :- *(2) Project [seller_id#206, price#208, freight_value#209, (price#208 + freight_value#209) AS gross#2312] : +- *(2) Filter ((isnotnull(price#208) AND (price#208 > 0.0)) AND isnotnull(seller_id#206)) : +- *(2) ColumnarToRow : +- FileScan parquet [seller_id#206,price#208,freight_value#209] Batched: true, DataFilters: [isnotnull(price#208), (price#208 > 0.0), isnotnull(seller_id#206)], Format: Parquet, Location: InMemoryFileIndex(1 paths)[file:/home/albakhtin/tiabd/Албахтин\IB\Spark\data\spark_input_parquet/o..., PartitionFilters: [], PushedFilters: [IsNotNull(price), GreaterThan(price,0.0), IsNotNull(seller_id)], ReadSchema: struct<seller_id:string,price:double,freight_value:double> +- BroadcastExchange HashedRelationBroadcastMode(List(input[0, string, false]),fals...	Disk Memory Serialized 1x Replicated	2	100.00%	2.6 MiB	0.0 B

Рисунок 6 – Кэш самостоятельного варианта В в живом Storage UI

На рисунке 7 представлен результат анализа.

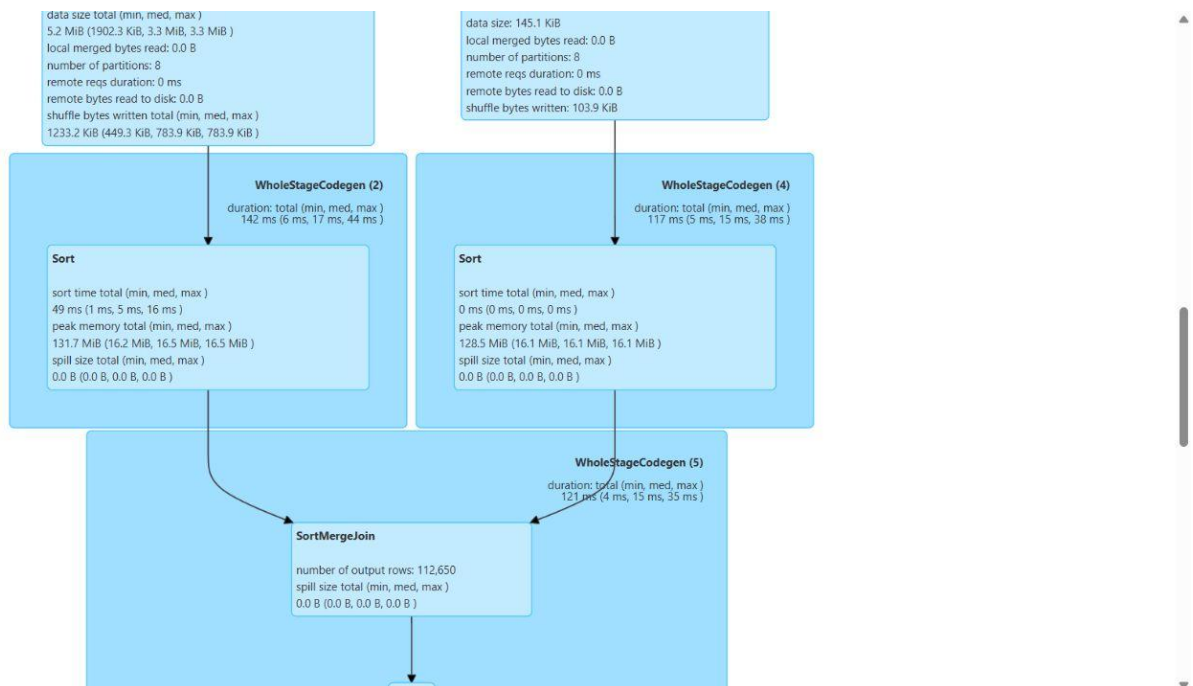


Рисунок 7 – План самостоятельного JOIN по seller_id

На рисунке 8 представлен результат анализа.

Details for Job 98

Status: SUCCEEDED
Submitted: 2026/10/05 19:54:23
Duration: 0.2 s
Associated SQL Query: 73
Job Group: P4_B_SMJ
Completed Stages: 4

Event Timeline
DAG Visualization

Completed Stages (4)

Page: 1

Stage Id	Description	Submitted	Duration	Tasks: Succeeded/Total	Input	Output	Shuffle Read	Shuffle Write
217	P4_B_SMJ count at <unknown>:0	2026/10/05 19:54:23	4 ms	1/1			472.0 B	
216	P4_B_SMJ count at <unknown>:0	2026/10/05 19:54:23	0.1 s	8/8			1337.1 KiB	472.0 B
215	P4_B_SMJ count at <unknown>:0	2026/10/05 19:54:23	34 ms	2/2	941.8 KiB			1233.2 KiB
214	P4_B_SMJ count at <unknown>:0	2026/10/05 19:54:23	22 ms	1/1	128.7 KiB			103.9 KiB

Page: 1

Рисунок 8 – Фактически завершённые стадии самостоятельного варианта В

9 Выводы и ограничения

1. Transformations описывают вычисления; Jobs появляются после action, что подтверждено отдельными журналами основной и самостоятельной серий.

2. Число входных разделов и SQL shuffle partitions различается; задачи следует связывать со стадиями физического плана.

3. Repartition изменяет размещение строк через Exchange, coalesce сохраняет особенности исходной нагрузки. Одинаковое число разделов не означает равномерность.

4. SMJ и BHI сохранили количество фактов; изменение стратегии меняет сортировку, передачу справочника и обмен по ключу.

5. Пользу persist следует оценивать с учётом заполнения кэша и всех повторных действий; отдельный быстрый повтор недостаточен для вывода об окупаемости.

6. Медианы трёх повторов зависят от р, прогрева и ресурсов VM; рост числа разделов не является универсальным ускорением.

7. В варианте В справочник sellers покрывает факты order_items. Группировка описывает штат продавца и не подменяет его штатом покупателя.

8. Вариант В повторяет ленивые операции, repartition/coalesce, обе стратегии JOIN, persist, четыре значения р и запись с двумя размерами разбиения.

9. Число и размеры part-файлов подтверждены файловой системой после write; coalesce(1) ограничивает параллелизм и требует обоснования размером результата.

Ограничения: локальная VM, два vCPU, короткие измерения, прогрев JVM и файловый кэш. Медиана уменьшает влияние выбросов, но не исключает фоновые процессы и не моделирует сеть промышленного кластера.

Ответы на контрольные вопросы

1. Чем transformation отличается от action с точки зрения момента фактического выполнения?

Transformation описывает новый план без обязательного вычисления. Action требует результат и запускает необходимые стадии выполнения.

2. Что означает lazy evaluation и почему замер времени вокруг withColumn() без Action некорректен?

Вычисления откладываются до action. Время withColumn без action измеряет построение выражения, а не обработку строк; для проверки вычисляемого поля нужен action, использующий его значение.

3. Как связаны Job, Stage и Task?

Job представляет выполнение действия, Stage объединяет задачи без необходимого промежуточного shuffle, Task обычно обрабатывает одну партицию стадии. Один action может породить несколько Jobs.

4. Почему число Tasks в разных Stages одного Job может отличаться?

До shuffle используются входные партиции, после него — число разделов нового распределения. Финальная глобальная агрегация может выполняться одной задачей.

5. Что такое partition и чем он отличается от физического CPU core?

Partition является частью данных, CPU core — аппаратным ресурсом. Один core последовательно выполняет задачи нескольких партиций; партиция не закреплена за ядром навсегда.

6. Почему local[*] сохраняет модель распределённого выполнения, хотя используется один компьютер?

Local[*] сохраняет планирование стадий, задачи и shuffle, но выполняет их потоками одной JVM. Сетевую масштабируемость такой запуск не проверяет.

7. Что называют narrow и wide transformation? Приведите примеры.

Narrow-операции используют ограниченное число входных разделов: filter, select, простые withColumn. Wide-операции перераспределяют данные: groupBy, repartition и многие JOIN.

8. Что такое shuffle и почему он обычно дороже локального filter()/select()?

Shuffle перераспределяет записи между разделами с записью и чтением промежуточных данных. Filter и select обычно работают локально; в кластере к цене shuffle добавляется сетевой обмен.

9. Как оператор Exchange в physical plan связан с shuffle?

Exchange обозначает изменение распределения данных. В зависимости от вида это shuffle либо broadcast, поэтому следует читать тип оператора, а не только его название.

10. В чём различие repartition() и coalesce()?

Repartition может увеличить или уменьшить число разделов и обычно выполняет shuffle. Coalesce преимущественно уменьшает число разделов объединением существующих без нового полного обмена.

11. Почему `coalesce()` может сохранить неравномерные размеры `partitions`?

Объединение сохраняет расположение исходных строк. Если крупные исходные партиции попадут в одну группу, дисбаланс останется или усилится.

12. Что такое `data skew` и как он проявляется на `geolocation_state`?

`Data skew` — неравномерность объёма данных по ключам или партициям. SP содержит около 40,42% `geolocation`, поэтому `hash`-разбиение по штату может перегрузить отдельные задачи.

13. Что задаёт `spark.sql.shuffle.partitions` и почему это не то же самое, что число `partitions` исходного `Parquet`?

Параметр задаёт стандартное число `shuffle partitions` для SQL-операций. Входные `Parquet partitions` зависят от файлов, их размеров и параметров чтения; на стенде `geolocation` читается в двух разделах.

14. Какие `physical operators` необходимо уметь распознавать в `explain("formatted")`?

Нужно различать `Scan`, `Filter`, `Project`, `Exchange`, `HashAggregate`, `Sort`, `SortMergeJoin`, `BroadcastExchange` и `BroadcastHashJoin`. При `persist` полезны `InMemoryTableScan`, при AQE — `AdaptiveSparkPlan` и `AQEShuffleRead`.

15. Почему `Sort Merge Join` часто содержит `Exchange` и `Sort` с обеих сторон?

Обе стороны должны быть распределены по ключу соединения и отсортированы внутри разделов. Уже подходящая раскладка может устранить часть этих действий.

16. При каком условии `Broadcast Hash Join` может быть выгоден?

Если малая сторона помещается в память каждого исполнителя, её `broadcast` позволяет соединять большую сторону локально без `shuffle` по ключу JOIN.

17. Почему большую таблицу нельзя безусловно broadcast-ить?

Broadcast создаёт копию таблицы у исполнителей и hash-структуру. Слишком большой объём может исчерпать память и увеличить задержку передачи или привести к тайм-ауту.

18. Почему одинаковое число строк результата важно при сравнении двух JOIN-стратегий?

Это базовая проверка логической эквивалентности: нельзя сравнивать скорость запросов, возвращающих разные факты. Дополнительно проверяются ключи и уникальность справочника.

19. Что происходит при первом Action после persist()?

Первое действие вычисляет lineage и заполняет зарегистрированный кэш. Сам persist только объявляет сохранение; стоимость материализации должна входить в сравнение.

20. Почему cache() может ухудшить одноразовую задачу?

Одноразовая задача получает дополнительные расходы на запись и хранение кэша без последующего повторного использования. На стенде даже двух действий оказалось недостаточно для окупаемости.

21. Для чего нужен unpersist()?

Unpersist освобождает сохранённые блоки. Blocking=True позволяет дожидаться освобождения перед следующей серией экспериментов.

22. Почему единичный benchmark-run недостаточен и зачем использовать медиану?

Прогрев JVM, JIT, дисковый кэш и фоновые процессы меняют время отдельных запусков. Медиана нескольких повторов менее чувствительна к отдельному выбросу, но не заменяет контроль условий.

23. Какие факторы делают абсолютное время Spark-задачи несопоставимым между разными компьютерами?

Влияют CPU, память, виртуализация, диски, версии Java и Spark, параметры сессии и фоновая нагрузка. Сравнивать следует варианты одного контролируемого стенда.

24. Почему слишком большое число partitions увеличивает overhead?

Возникает больше событий планировщика, сериализации, служебных объектов и мелких задач. Их расходы могут превысить выигрыш параллелизма.

25. Как связаны output partitions, Tasks записи и число part-файлов?

Выходная задача обычно записывает файл своей партии. Пустые разделы, разделение записи по каталогам и настройки формата могут нарушить точное равенство; число файлов нужно измерять.

26. Почему coalesce(1) не является универсальным способом борьбы с small files?

Один раздел устраняет параллелизм записи и концентрирует данные в одном потоке. Размер набора и дальнейшее чтение важнее самого желания иметь один файл.

27. Что меняет Adaptive Query Execution и почему основные опыты этой работы выполняются с AQE=false?

AQE использует фактическую статистику, объединяет мелкие shuffle partitions и может менять стратегию JOIN. AQE=false фиксирует условия основных опытов и упрощает сравнение.

28. Какие вкладки Spark UI наиболее полезны для анализа groupBy, JOIN и cache?

Jobs связывает действие с выполнением; Stages показывает задачи и shuffle; SQL/DataFrame раскрывает физический план; Storage подтверждает материализацию и освобождение кэша.

29. В чём различие между логическим выводом о данных и экспериментальным выводом о вычислительной системе?

Логический вывод описывает данные, например выручку штата. Вычислительный вывод описывает цену конкретного плана при фиксированных данных и конфигурации.

30. Как сформулировать корректный вывод о производительности так, чтобы не превратить локальный результат в универсальное правило?

Следует назвать данные, оборудование и параметры, изменённый фактор, число повторов, медиану и подтверждение в плане/UI. Вывод ограничивается этими условиями, а не распространяется на все кластеры.

Источники и приложения

Предоставленная методичка практики №4. Новые Practice_4.ipynb, results/pr4.json, снимки REST API, физические планы и агрегаты Parquet приложены в личном каталоге Spark. Apache Spark 3.5.7: <https://spark.apache.org/docs/3.5.7/monitoring.html> и <https://spark.apache.org/docs/3.5.7/tuning.html>.